

CSE 312 İşletim Sistemleri

Senkronizasyon (Synchronization)

Kapsamlı Konu Anlatımı & Soru Bankası

Ders	CSE 312 – İşletim Sistemleri (Operating Systems)
Dönem	Spring 2026
Konu	Lecture 05 – Synchronization
Kaynak	Ders slaytları + Modern Operating Systems 5th Ed. (Tanenbaum)

İçindekiler

1. Zamanlayıcılar (Timers) ve Sistem Saati
2. Klasik Senkronizasyon Problemleri
3. Yemek Yiyen Filozoflar (Dining Philosophers)
4. Okuyucular ve Yazarlar (Readers & Writers)
5. Deadlock (Kilitlenme)
6. Deadlock'tan Kaçınma – Banker's Algorithm
7. Deadlock'u Önleme (Prevention)
8. İki Fazlı Kitleme ve Gerçek Hayat
9. Açık Uçlu Quiz Soruları ve Cevapları
10. Kitap Bölüm Sonu Soruları (İngilizce + Türkçe) ve Çözümleri

1. Zamanlayıcılar (Timers) ve Kernel Senkronizasyonu

Linux çekirdeğinde zaman yönetimi, senkronizasyon ile iç içe geçmiştir. Özellikle jiffies değişkeni ve gerçek zamanlı saat (xtime) gibi paylaşılan yapılar, eş zamanlı erişim sorunları yaratır.

1.1 Jiffies ve 64-bit Sorun

Her zamanlayıcı kesmesinde (timer tick) bir sayaç olan jiffies artar. 32-bit makinelerde, 64-bit bir değere atomik erişim mümkün değildir. Bu nedenle kernel, jiffies_64 yapısını bir spin-lock ile korur.

⚠ Neden 64-bit Değer 32-bit Makinede Sorun Çıkarır?

- 32-bit makine, 64-bit değeri iki ayrı hafıza operasyonu ile okur.
- İki okuma arasında bir kesme (interrupt) devreye girerse tutarsız değer okunabilir.
- Bu atomiklik (atomicity) sorununa klasik bir örnektir.

Kernel, jiffies değişkenini jiffies_64'ün düşük anlamlı 32 biti olarak tanımlamak için linker magic kullanır. Böylece sadece bir değişken güncellenir; ancak 64-bit değere erişim için get_jiffies_64() fonksiyonu çağrılır ve spin-lock koruması devreye girer.

1.2 Kernel Zamanlayıcıları

Zamanlayıcı Türü	Amaç	Kullanım Alanı
Dynamic Timer	Belirli bir süre sonra bir fonksiyon çağır	Ağ time-out, I/O gecikmesi
Delay Loop	Çok kısa süreli meşgul bekleme (busy-wait)	Microsecond/nanosecond gecikmeler
Watchdog Timer	Sistemin donmaması için düzenli "reset" sinyali	Gömülü (embedded) sistemler

i Watchdog Timer Mantığı

- Sistem düzgün çalışırken Watchdog sayacını sıfırlar (reset atar).
- Sistem donduğunda sayaç dolmaya devam eder.
- Sayaç taşıdığı donanım sistemi otomatik yeniden başlatır.
- Uzaktan erişilemeyen gömülü sistemlerde hayati önem taşır.

2. Klasik Senkronizasyon Problemleri

Senkronizasyon primitiflerini anlamak için kullanılan bu problemler, gerçek dünya uygulamalarındaki ilkeleri öğretmek amacıyla bilinçli olarak tasarlanmış soyut senaryolardır.

i Klasik Problemlerin Amacı

- Senkronizasyon primitiflerini (semaphore, mutex, monitor) somutlaştırmak
- Deadlock, starvation gibi sorunları göstermek
- Gerçek çözümlerin takip etmesi gereken ilkeleri ortaya koymak

3. Yemek Yiyen Filozoflar (Dining Philosophers)

Beş filozof bir masada oturmakta; her birinin önünde makarna tabağı, aralarında birer çatal bulunmaktadır. Yemek yiyebilmek için bir filozofun iki çatala ihtiyacı vardır (sol ve sağ). Filozoflar ya düşünür ya da yer.

3.1 Kötü Çözüm - Deadlock

Aşağıdaki koda göre her filozof önce sol çatalı, sonra sağ çatalı almaya çalışır:

```
void philosopher(int i) {
    while(TRUE) {
        think();
        take_fork(i);          // sol catal
        take_fork((i+1) % N);  // sag catal
        eat();
        put_fork(i);
        put_fork((i+1) % N);
    }
}
```

❗ Neden Deadlock Oluşur?

Tüm filozoflar aynı anda sol çatalı alırsa her birinin elinde 1 çatal kalır.

Kimse sağ çatalı alamaz → Herkes bekler → Sistem kilitlenir (deadlock).

3.2 Mutex ile Basit Çözüm - Ama Yetersiz

Çatal almadan önce bir mutex kilitlenir. Bu şekilde aynı anda yalnızca bir filozof çatal alabilir ve deadlock engellenir. Ancak bu çözümün sorunu: aynı anda yalnızca bir kişi yiyebilir → performans düşer.

⚠️ Mutex'i Yemekten Önce Bırakmak İşe Yaramaz

E filozofu mutex'i bırakıp yemeğe başlarsa; A, B, C, D tekrar yarışa girer.

Problem başa döner: biri çatalı alır, diğerleri bekler → olası kilitlenme.

3.3 İyi Çözüm - Durum Makinesi + Koşullu Uyandırma

Her filozofun üç durumu vardır: THINKING, HUNGRY, EATING. Temel ilkeler:

- Çatal alırken mutex kilitlenir.
- Her iki çatal da müsaitse yemeğe geçilir; değilse HUNGRY kalınır ve uyutulur (sleep).
- Çatalklar bırakıldığında komşular HUNGRY ise uyandırılır (wake).
- Her semaphore girişinde kuyruk (queue) varsayımı gerekli.

✓ Çözümün Doğruluğu (Sketch of Proof)

E yiyerken: A ve D yiyemez. B ile C'den en fazla biri yiyebilir.

E bitirince: A ve D uyandırılır. D hâlâ bloklı, A çalışabilir.

C bitirince: B çalışabilir.

Bu durum ancak yemenin sonlu sürdüğü ve semaphore'larda FIFO kuyruğu olduğu varsayımıyla doğrudur.

3.4 Rastgelelik (Randomization) ile Çözüm

İkinci çatalı alamayan filozof rastgele bir süre uyur ve tekrar dener. Pratikte çalışır; ancak garanti verilmez. Gerçek zamanlı sistemler için yetersizdir. Knuth'un hash tabloları için verdiği uyarıyla örtüşür: "Ortalama verimli ama en kötü durum korkunç."

4. Okuyucular ve Yazarlar (Readers and Writers)

Birden fazla süreç ortak bir veritabanına erişmektedir. Aynı anda birden fazla okuyucu olabilir; ancak yazma sırasında başka okuyucu ya da yazar bulunamaz.

Durum	İzin Verilir mi?
N okuyucu aynı anda okur	✓ Evet
1 yazıcı + 0 okuyucu/yazıcı	✓ Evet
1 yazıcı + herhangi okuyucu	✗ Hayır
2+ yazıcı aynı anda	✗ Hayır

4.1 Çözüm Taslağı

- rc (reader count): kaç okuyucu var, mutex ile korunur.
- İlk okuyucu gelince veritabanı mutex'i kilitler.
- Sonraki okuyucular sadece rc'yi artırır.
- Son okuyucu gidince veritabanı mutex'ini serbest bırakır.
- Yazar gelince veritabanı mutex'ini bekler.

⚠ Starvation Tehlikesi

Okuyucular sürekli gelirse yazar sonsuza kadar bekleyebilir → Writer starvation.

Çözüm: Yazar bekliyorsa, yeni gelen okuyucu yazarın arkasına eklenir.

Bu değişiklik eşzamanlılığı düşürür (concurrency↓) ama adaleti artırır.

4.2 Dersler

Basit mutex yetmez	Genellikle birden fazla senkronizasyon primitifi gerekir.
Karmaşık algoritmalar	Yazması ve doğruluğunu kanıtlaması zordur.
Adalet (Fairness)	Starvation ve indefinite overtaking asıl sorun olabilir.
Deadlock takibi	Her çok-kaynak senaryosunda deadlock tehlikesi gözetilmelidir.

5. Deadlock (Kilitlenme)

Alice tarayıcıyı tutmakta ve yazıcıyı beklemektedir. Bob ise yazıcıyı tutmakta ve tarayıcıyı beklemektedir. İkisi de ilerleme kaydedemez → klasik deadlock senaryosu.

5.1 Coffman Koşulları (1971)

Bir deadlock oluşabilmesi için dört koşulun tamamının aynı anda gerçekleşmesi gerekir:

1. Mutual Exclusion (Karşılıklı Dışlama)	Her kaynak ya boş ya da tek bir süreçte atanmış olabilir.
2. Hold and Wait (Tut ve Bekle)	Kaynakları tutan süreçler ek kaynak isteyebilir.
3. No Preemption (Geri Alma Yok)	Kaynaklar zorla geri alınamaz; süreç gönüllü bırakmalıdır.
4. Circular Wait (Döngüsel Bekleme)	İki veya daha fazla süreçten oluşan, her birinin bir sonrakinin beklediği döngüsel zincir.

⚡ Kilit Nokta

Bu dört koşuldaki HERHANGİ BİRİ sağlanmıyorsa deadlock OLUŞAMAZ.

Deadlock önleme stratejileri bu koşulları ortadan kaldırmaya çalışır.

5.2 Yönlendirilmiş Graf ile Modelleme

Deadlock, kaynak tahsisat grafi ile görselleştirilebilir:

Yay Yönü	Anlamı
Kaynak → Süreç	Kaynak bu sürece atanmış (allocated)
Süreç → Kaynak	Süreç bu kaynağı bekliyor (waiting)
Grafta döngü var	Deadlock mevcut (veya olası)

5.3 Deadlock ile Başa Çıkma Stratejileri

Strateji	Yöntem	✓ Avantaj	✗ Dezavantaj
Yoksay (Ignore)	Olasılık düşükse hiçbir şey yapma	Basit; çoğu OS bunu yapar	Seyrek de olsa sistem çöker
Tespit + Kurtarma (Detect & Recover)	Grafı analiz et, döngü bulunca müdahale et	Kesin tespit	Yüksek maliyet, kurtarma zor
Kaçınma (Avoid)	Banker's Algorithm ile güvenli durum takibi	Garantili deadlock yok	Tüm kaynak taleplerinin önceden bilinmesi gerekir
Önleme (Prevent)	4 koşuldan birini yok et	Kalıcı çözüm	Performans kaybı veya esneklik azalır

5.4 Deadlock Tespit Algoritması (Çok Kaynaklı)

Birden fazla türden kaynak olduğunda (örn: 3 yazıcı, 2 tarayıcı) aşağıdaki matris-vektör yöntemi kullanılır:

Sembol	Anlamı
E (Existing)	Her kaynak türünün toplam sayısı
A (Available)	Şu an boş olan kaynak sayısı
C (Current)	Her sürecin mevcut tahsisatı ($n \times r$ matris)
R (Request)	Her sürecin hâlâ istediği kaynaklar ($n \times r$ matris)

Algoritma adımları:

1. Tüm süreçleri "işaretlenmemiş" başlat (işaretli = tamamlanabilir).
2. $R_i \leq A$ olan işaretlenmemiş bir P_i süreç bul (isteği şu an karşılanabilir).
3. Bulunduyorsa $A \leftarrow A + C_i$ yap (bu süreç bitince kaynakları serbest kalır) ve 2'ye dön.
4. Hiç böyle süreç yoksa, işaretlenmemiş süreçler deadlock içindedir.

5.5 Örnek - Algoritmanın Çalıştırılması

Sistem: 4 tape drive (TD), 2 plotter (P), 3 scanner (S), 1 CD-ROM (CD). $E = (4, 2, 3, 1)$. Üç süreç mevcut ve $A = (2, 1, 0, 0)$.

Çözüm Adımları

- ① P3 çalıştırılabilir ($R_3 \leq A$). Bittikten sonra $A = (2,2,2,0)$
 - ② P2 artık çalıştırılabilir. Bittikten sonra $A = (4,2,2,1)$
 - ③ P1 artık çalıştırılabilir: $(2,0,0,1) \leq (4,2,2,1)$. ✓ Deadlock yok!
- Eğer P3'ün isteği $(2,1,1,1)$ olsaydı: $(1,0,1,1) \leq (2,1,0,0)$ sağlanmaz → Deadlock!

6. Güvenli/Güvensiz Durumlar ve Banker's Algorithm

6.1 Güvenli ve Güvensiz Durum

Durum	Tanım
Güvenli (Safe)	Tüm süreçlerin maksimum taleplerini karşılayabilecek en az bir çizelgeleme sırası mevcuttur.
Güvensiz (Unsafe)	Böyle bir sıra yoktur; ancak henüz deadlock olmayabilir.
Deadlock	Hiçbir süreç ilerleyemez.

Güvensiz bir durum her zaman deadlock'a yol açmaz; süreçler maksimum taleplerinden daha azını kullanırsa sistem yine de kurtulabilir. Ancak OS garanti veremez.

6.2 Banker's Algorithm (Dijkstra, 1965)

Önce tek kaynak türü için düşünelim: bir bankacının kasasında sınırlı para var; her müşterinin bir kredi limiti var. Bankacı, müşterilerin borçlarını güvenli sırada kapatabilecek şekilde kredi verir.

```
# Tek kaynak – örnek
Has  Max
A: 1   6  ← A'ya 1 daha verilirse Has=2, kalan free=1
B: 1   5  ← Güvensiz! B veya D ihtiyacı karşılanamaz
C: 2   4
D: 4   7
Free: 2
```

Çok Kaynaklı Banker's Algorithm:

1. R matrisinin işaretli bir satırı için $R_i \leq A$ koşulunu kontrol et.
2. Sağlanıyorsa satırı işaretle; $A \leftarrow A + C_i$.
3. Tüm satırlar işaretlenirse durum güvenli; aksi hâlde güvensiz.

❏ Neden Pratikte Kullanılmıyor?

- Her sürecin başlangıçta kaynak taleplerini belirtmesi gerekir → pratikte imkânsız.
- Süreçler dinamik olarak gelir ve gider.
- Donanım arızaları kaynakları yok eder.
- Performans maliyeti yüksektir.

7. Deadlock'u Önleme (Prevention)

Coffman'ın dört koşulundan en az birini kalıcı olarak ortadan kaldırarak deadlock'u önleyebiliriz.

Hedef Koşul	Yöntem	Kısıt/Sorun
Mutual Exclusion'a Saldır	Yazıcılar spooled → sadece daemon talep eder → deadlock riski azalır.	Tüm kaynaklar spoolanamaz (örn: tape drive).
Hold & Wait'e Saldır	Süreç başlangıçta tüm kaynakları bildirir. Veya bir kaynağa ihtiyaç varken tuttuklarını bırak.	Önceden bilmek genellikle mümkün değil; kaynak israfı artar.
No Preemption'a Saldır	Kaynak geri alınabilirse (örn: CPU, RAM sayfası) zorla al.	Yazıcı gibi cihazlar yarı iş bırakılamaz.
Circular Wait'e Saldır	Kaynakları numaralandır; her süreç artan sırada talep etsin.	Eski mainframe sistemlerde uygulandı; bugün sınırlı kullanım.

7.1 İki Fazlı Kilitleme (Two-Phase Locking)

Veri tabanlarında sıklıkla kullanılan bu yöntemde süreç iki fazda çalışır:

Faz 1: Gereken tüm kayıtları kilitlemeye çalış. Herhangi birini kilitlenemezse tüm kilitleri bırak ve baştan başla.

Faz 2: Tüm kilitler alındıktan sonra güncellemeleri yap ve kilitleri serbest bırak.

i Gerçek Hayatta Deadlock

- İşletim sistemleri deadlock için genellikle hiçbir şey yapmaz (ignore).
- Bazı kaynakları (process slot gibi) fazladan sağlar (overprovision).
- Veri tabanları two-phase locking ve timeout mekanizmaları kullanır.
- Fork bomb: süreç kendini sürekli kopyalayarak proses tablosunu doldurur → deadlock değil ama kaynak tükenmesi.

8. Süreç Yörüngesi ve Güvenli Durum (Process Trajectories)

İki sürecin kaynak kullanımı, iki boyutlu bir uzayda yörünge olarak çizilebilir. Tehlikeli bölgeler (deadlock'a gidecek bölgeler) grafta kırmızı sınırlı alanlar olarak gösterilir.

Bölge	Anlamı
Yeşil	Her iki süreç de yazıcıyı tutuyor → imkânsız bölge
Sarı	Her iki süreç de plotter'ı tutuyor → imkânsız
Mavi	Her iki süreç de ikisini tutuyor → tamamen imkânsız
Kırmızı sınır	Bu alana girilirse deadlock kaçınılmaz

OS, bir kaynak tahsisatının bu tehlikeli bölgeye girip girmeyeceğini kontrol ederek (Banker's Algorithm ile) güvenli olmayan tahsisatları geciktirir.

✓ Önemli Çıkarım: Güvensiz $\neq$ Deadlock

Güvensiz bir durumda bile süreçler erken kaynak bırakırsa deadlock olmayabilir.

OS garantiyi ancak güvenli durumda verebilir.

Deadlock avoidance = her adımda güvenli durum garantisi.

9. Açık Uçlu Quiz Soruları ve Cevapları

Aşağıdaki sorular sınav formatında hazırlanmıştır. Her soru, ders notlarında ve kitapta ele alınan kavramları kapsamaktadır.

S1. Linux çekirdeğinde jiffies_64 değişkenine neden doğrudan erişilmez? 64-bit değer atomik olarak okunamama sorunu nasıl çözülmüştür?

S1. Linux çekirdeğinde jiffies_64 değişkenine neden doğrudan erişilmez? 64-bit değerin atomik olarak okunamama sorunu nasıl çözülmüştür?

32-bit bir makinede, 64-bit veri iki ardışık bellek işlemiyle okunur. Bu iki işlem arasında bir kesme (interrupt) gerçekleşirse tutarsız bir değer okunabilir. Kernel bu sorunu çözmek için:

- ① spin-lock ile jiffies_64'e erişimi senkronize eder,
- ② get_jiffies_64() fonksiyonunu sağlar,
- ③ jiffies değişkenini linker magic ile jiffies_64'ün düşük 32 biti olarak tanımlar (böylece sadece bir değişken her tick güncellenir).

S2. Yemek Yiyen Filozoflar probleminde tüm filozofların aynı anda sol çatalı almasına izin veren bir algoritmanın neden deadlock'a yol açtığını Coffman koşulları çerçevesinde açıklayın.

S2. Yemek Yiyen Filozoflar probleminde tüm filozofların aynı anda sol çatalı almasına izin veren bir algoritmanın neden deadlock'a yol açtığını Coffman koşulları çerçevesinde açıklayın.

- ① Mutual exclusion: her çatal aynı anda en fazla bir filozofta olabilir.
 - ② Hold and wait: her filozof sol çatalı tutarken sağ çatalı bekler.
 - ③ No preemption: çatalar zorla alınamaz.
 - ④ Circular wait: A sağ çatalı için B'yi, B → C, C → D, D → E, E → A'yı bekler.
- Tüm dört koşul sağlandığından deadlock kaçınılmazdır.

S3. Okuyucular-Yazarlar probleminde "okuyucu öncelikli" çözüm ile "yazar öncelikli" çözüm arasındaki temel farkı ve her birinin getirdiği tehlikeyi açıklayın.

S3. Okuyucular-Yazarlar probleminde "okuyucu öncelikli" çözüm ile "yazar öncelikli" çözüm arasındaki temel farkı ve her birinin getirdiği tehlikeyi açıklayın.

Okuyucu öncelikli: Yazar beklerken gelen yeni okuyucular kabul edilir. Tehlike: Sürekli gelen okuyucular yazarı sonsuza bekletir (writer starvation).

Yazar öncelikli: Yazar bekliyorken gelen yeni okuyucular yazarın arkasına eklenir. Tehlike: Eşzamanlılık düşer (performans azalır); okuyucu starvation riski doğabilir.

S4. Banker's Algorithm neden günümüzde pratikte kullanılmıyor? Hangi varsayımları gerçek sistemlerde karşılanamaz?

S4. Banker's Algorithm neden günümüzde pratikte kullanılmıyor? Hangi varsayımları gerçek sistemlerde karşılanamaz?

- ① Her sürecin başlangıçta maksimum kaynak gereksinimini bildirmesi gerekir (pratikte bilinmez).
- ② Süreçler dinamik olarak gelir ve gider; statik analiz geçersiz kalır.
- ③ Donanım arızaları kaynakları beklenmedik şekilde ortadan kaldırır.
- ④ Algoritmanın her kaynak talebinde çalıştırılması performans maliyeti yaratır.

S5. "Circular wait" koşulunu ortadan kaldırmak için kaynakları numaralandırma yöntemi nasıl çalışır? Neden deadlock'u önler?

S5. "Circular wait" koşulunu ortadan kaldırmak için kaynakları numaralandırma yöntemi nasıl çalışır? Neden deadlock'u önler?

Kaynaklar 1'den n'e kadar numaralandırılır. Her süreç kaynakları yalnızca artan sayı sırasıyla talep edebilir. Döngüsel bekleme için $A \rightarrow B \rightarrow \dots \rightarrow A$ zinciri gerekir; ancak sıralı talep kuralı gereği her süreç bir öncekinden daha yüksek numaralı kaynağı bekler. Bu nedenle döngü oluşamaz, Coffman'ın 4. koşulu sağlanamaz.

S6. Deadlock kurtarma yöntemlerini karşılaştırın: preemption, checkpoint/rollback ve süreç öldürme. Her birinin en uygun kullanım senaryosunu belirtin.

S6. Deadlock kurtarma yöntemlerini karşılaştırın: preemption, checkpoint/rollback ve süreç öldürme. Her birinin en uygun kullanım senaryosunu belirtin.

Preemption: Kaynaklar zorla alınabiliyorsa (örn: RAM sayfaları) mümkündür. Uygun: CPU veya bellek gibi preempt edilebilir kaynaklar.
Checkpoint/rollback: Süreçlerin durumu periyodik olarak kaydedilir; deadlock oluşunca bir süreç geri sarılır. Uygun: Veritabanı işlemleri.
Süreç öldürme: En basit yöntem; kaynağı tutan bir süreç öldürülür. Uygun: Yeniden çalıştırılabilir batch işlemler.

S7. İki fazlı kilitleme (two-phase locking) deadlock'u nasıl önler? Hangi dezavantajı vardır?

S7. İki fazlı kilitleme (two-phase locking) deadlock'u nasıl önler? Hangi dezavantajı vardır?

Faz 1'de tüm kilitler tek tek alınmaya çalışılır; herhangi biri alınamazsa tüm kilitler bırakılır ve baştan başlanır. Bu sayede "hold and wait" koşulu etkili biçimde kırılır: ya tüm kaynaklar alınır ya da hiçbiri tutulmaz.

Dezavantaj: Aynı süreçler sürekli birbirinin kilidini beklerse livelock (canlı kilitleme) riski doğar; algoritma tekrar tekrar baştan başlayabilir.

S8. "Fork bomb" nedir ve bir işletim sisteminin bu saldırıya karşı nasıl önlem alabileceğini açıklayın.

S8. "Fork bomb" nedir ve bir işletim sisteminin bu saldırıya karşı nasıl önlem alabileceğini açıklayın.

Fork bomb: Bir süreç sürekli kendini kopyalayarak (fork()) proses tablosunu doldurur ve sistem kaynaklarını tüketir; bu bir DoS (Denial of Service) saldırısıdır.

Önlemler:

- ① Kullanıcı başına maksimum proses sayısı sınırı (ulimit -u).
- ② Process slot'larını fazladan sağlama (overprovision).
- ③ Cgroups ile kaynak sınırlandırma (Linux).

10. Kitap Bölüm Sonu Soruları ve Çözümleri

Aşağıdaki sorular Modern Operating Systems, 5th Ed. (Tanenbaum) Bölüm 2'nin bölüm sonu sorularından senkronizasyon konusuyla ilgili olanlar seçilmiştir. Her sorunun İngilizce orijinali ve Türkçe çevirisi verilmekte; çözümler yalnızca Türkçe olarak sunulmaktadır.

Soru 23

EN: Does Peterson's solution to the mutual-exclusion problem shown in Fig. 2-24 work when process scheduling is preemptive? How about when it is nonpreemptive?

TR: Şekil 2-24'te gösterilen Peterson çözümü, süreç planlaması öncel (preemptive) olduğunda çalışır mı? Öncel olmayan (nonpreemptive) planlamada ne olur?

► Çözüm

Preemptive planlamada: Evet, Peterson'ın çözümü doğru çalışır. `turn`, `interested[]` gibi paylaşılan değişkenlere erişimler atomik değildir; ancak algoritma, bir süreç kesilse bile dışı bölgede (critical section) birden fazla sürecin yer almamasını garanti eder. Atomik okuma/yazma varsayımı yeterlidir.

Nonpreemptive planlamada: Yine doğru çalışır. Süreçlerin yarışmalı güncelleme yaptığı tek çekirdekli sistemde, tek bir süreç CPU'yu kullandığında kesme yoktur; bu aslında daha kısıtlı bir ortamdır ve algoritmanın gerektirdiği koşullar sağlanır.

Soru 27

EN: What is a race condition?

TR: Yarış koşulu (race condition) nedir?

► Çözüm

Race condition, iki veya daha fazla sürecin ya da iş parçacığının paylaşılan veriye eş zamanlı erişimi sırasında çıktının bu erişimlerin sırasına bağlı olduğu duruma denir. Yani sonuç, kimin "önce koştuğuna" göre değişir. Kritik bölge (critical section) mekanizmaları tam olarak bu durumu önlemek için tasarlanmıştır.

Soru 29

EN: The producer-consumer problem can be extended to a system with multiple producers and consumers that write (or read) to (from) one shared buffer. Assume that each producer and consumer runs in its own thread. Will the solution presented in Fig. 2-28, using semaphores, work for this system?

TR: Üretici-tüketici problemi, birden fazla üretici ve tüketicinin ortak bir tampon kullandığı sisteme genişletilebilir. Şekil 2-28'deki semafor çözümü bu sistem için işe yarar mı?

► Çözüm

Evet, işe yarar. Şekil 2-28'deki çözümde:

- "empty" semaforu tampondaki boş slotları temsil eder.
- "full" semaforu dolu slotları temsil eder.
- "mutex" semaforu tampon indeksine (in, out) erişimi korur.

Birden fazla üretici veya tüketici olduğunda mutex, kritik bölgeye sırasıyla girmeyi sağlar. full/empty semaforları kaynak sayısını doğru şekilde izlediği için çözüm çok üretici-çok tüketici senaryosunda da tutarlı çalışır.

Soru 31

EN: Show how counting semaphores (i.e., semaphores that can hold an arbitrary value) can be implemented using only binary semaphores and ordinary machine instructions.

TR: Sayma semaforlarının (keyfi değer alabilen) yalnızca ikili semaforlar ve normal makine komutları kullanılarak nasıl uygulanabileceğini gösterin.

► Çözüm

Üç değişken kullanırız: count (tam sayı, sayma değeri), mutex (ikili semafor, count'u korur), delay (ikili semafor, engellenmiş süreçler için).

```
down(s):
down(mutex)
count -= 1
if count < 0: up(mutex); down(delay) // blokla
else: up(mutex)
```

```
up(s):
down(mutex)
count += 1
if count <= 0: up(delay) // bekleyeni uyandır
up(mutex)
```

Bu yapı; mutex ile count güncellemelerini atomik yapar, delay ile engellenmiş süreçleri bekletir.

Soru 32

EN: *If a system has only two processes, does it make sense to use a barrier to synchronize them? Why or why not?*

TR: Sistemde yalnızca iki süreç varsa, bunları senkronize etmek için bariyer kullanmak mantıklı mıdır? Neden?

► Çözüm

Evet, iki süreç için de bariyer mantıklı olabilir. Bariyer, tüm katılımcı süreçlerin belirli bir noktaya ulaşmasını bekler. İki süreç söz konusu olsa bile "her ikisi de hazır olduğunda devam et" semantiği gerekiyorsa bariyer doğal bir çözümdür.

Örnek: İki iş parçacığı matrisin farklı yarısını işliyor ve $n+1$. iterasyona yalnızca her ikisi de n 'yi tamamlayınca geçmelidir.

Ancak iki süreç için alternatiflerin (örn: join, semafor) daha hafif ve yeterli olabileceğini de belirtmek gerekir.

Soru 33

EN: *Can two threads in the same process synchronize using a kernel semaphore if the threads are implemented by the kernel? What if they are implemented in user space? Assume that no threads in any other processes have access to the semaphore.*

TR: Aynı süreçteki iki iş parçacığı, kernel semafor kullanarak senkronize olabilir mi? Bu iş parçacıkları kernel tarafından uygulandığında ve kullanıcı uzayında uygulandığında ne olur?

► Çözüm

Kernel-level threads: Evet. Kernel, her thread'i bağımsız bir çizelgeleme birimi olarak görür; kernel semafor down/up çağrıları system call ile yapılır ve kernel doğrudan müdahale edebilir. Etkili senkronizasyon sağlanır.

User-level threads: Teknik olarak evet, ama verimsiz. Kernel, kullanıcı iş parçacıklarının varlığından habersizdir. Bir thread semaforda bloklarsa kernel tüm süreci uyutabilir, diğer kullanıcı iş parçacığı da çalışamaz. Kullanıcı uzayı iş parçacıkları için kullanıcı-uzayı mutex/semafor tercih edilmelidir.

Soru 53

EN: The readers and writers problem can be formulated in several ways with regard to which category of processes can be started when. Carefully describe three different variations of the problem, each one favoring (or not favoring) some category of processes.

TR: Okuyucular-Yazarlar problemi, hangi süreç kategorisinin ne zaman başlayabileceğine göre farklı biçimlerde formüle edilebilir. Her biri farklı bir kategoriyi öne çıkaran üç değişikliği açıklayın.

► Çözüm

Değişiklik 1 - Okuyucu Öncelikli:

Yazar beklerken yeni okuyucular kabul edilir. Okuyucu sayısı > 0 ise yazar bekler. Tehlike: Writer starvation.

Değişiklik 2 - Yazar Öncelikli:

Yazar bekliyorken gelen yeni okuyucular yazarın arkasına eklenir. Aktif okuyucular bitince yazar çalışır, ardından bekleyen okuyucular çalışır. Tehlike: Reader starvation olası; eşzamanlılık azalır.

Değişiklik 3 - Adil (FIFO):

Hem okuyucular hem yazarlar geliş sırasına göre sıraya girer. Hiçbir kategori öncelik almaz. Tehlike: Eşzamanlı okuma fırsatları azalır (verimlilik düşer).

Soru 55

EN: Assume that you have an operating system that provides semaphores. Implement a message system. Write the procedures for sending and receiving messages.

TR: Semafor sağlayan bir işletim sisteminiz olduğunu varsayın. Mesaj sistemi uygulayın; mesaj gönderme ve alma prosedürlerini yazın.

► Çözüm

Ortak yapılar:

buffer[N] // N mesajlık dairesel tampon

in = out = count = 0

mutex = 1 // tampon indekslerini korur

empty = N // bos slot sayısı

full = 0 // dolu slot sayısı

send(msg):

down(empty) // bos yer bekle

down(mutex)

buffer[in] = msg

in = (in+1) % N

count++

up(mutex)

up(full) // alıcıları bilgilendir

receive() -> msg:

down(full) // mesaj bekle

down(mutex)

msg = buffer[out]

out = (out+1) % N

count--

up(mutex)

up(empty) // üreticiyi bilgilendir

return msg

Soru 57

EN: Write a producer-consumer problem that uses threads and shares a common buffer. However, do not use semaphores or any other synchronization primitives to guard the shared data structures. See how long it takes for a fatal race condition to occur.

TR: Ortak tampon kullanan üretici-tüketici problemi yazın; ancak paylaşılan veri yapılarını korumak için semafor ya da başka senkronizasyon primitifi kullanmayın. Ölümcül yarış koşulunun ne kadar sürede ortaya çıktığını gözlemleyin.

► Çözüm

Bu soru deneysel bir gözlem sorusudur. Beklenen analiz:

- Senkronizasyon olmadan üretici "in" indeksini, tüketici "out" indeksini güncellerken çakışma olabilir.
- Tipik sorunlar:
 - count değişkenine eş zamanlı erişim: okuma-artırma-yazma döngüsü bölünebilir.
 - Üretici taşmış tampona yazabilir (count=N iken ekleyebilir).
 - Tüketici boş tampondan okuyabilir.
- Modern çok çekirdekli sistemlerde yarış koşulu çok kısa sürede (genellikle milisaniyeler içinde) oluşabilir.
- Bu deney, kritik bölge korumasının neden zorunlu olduğunu somut biçimde gösterir.